

面向 Ansys Maxwell 的工业软件智能体构建与验证

一种由自然语言驱动的仿真规格生成、脚本执行与反馈修正方法

工业软件智能化项目阶段报告

2026 年 4 月 29 日

摘要

工业仿真软件通常功能强大，但使用门槛较高。用户需要理解建模、材料、激励、边界、求解器和后处理等多个环节，才能得到可信结果。本项目以 Ansys Maxwell 为对象，构建了一个工业软件智能体原型：用户输入自然语言需求后，系统将需求整理为仿真规格，生成可执行的 PyAEDT 脚本，在本地调用 Maxwell 完成建模、求解和结果提取，并根据结果判断需求是否满足。若关键约束未满足，系统会把仿真结果和失败原因反馈给大模型，由大模型生成下一轮修改方案，再重新仿真验证。

与简单的问答系统不同，本系统的输出不是文字建议，而是实际生成的 `.aedt` 工程文件、结构化结果和约束判定。当前已验证的任务族包括二维电磁铁、二维平行板电容器、二维变压器概念模型和二维电感器概念模型。实验表明，该方法能够把“中文需求”转化为“可运行的 Maxwell 仿真任务”，并在电压、电流等数值约束冲突时完成反馈修正。本文以通俗方式说明系统方法、实现流程、实验结果、复现步骤和现阶段边界。

关键词：工业软件智能体；Ansys Maxwell；PyAEDT；大模型；仿真自动化；反馈修正

目录

1	研究背景与问题	3
2	总体思路	3
2.1	为什么不直接让大模型生成 .aedt 文件	3
2.2	核心闭环	3
3	系统方法	4
3.1	自然语言到仿真规格	4
3.2	仿真规格到 PyAEDT 脚本	5
3.3	结果评价	5
3.4	反馈修正	6
4	已实现任务族	6
5	实验设计与结果	7
5.1	实验设置	7
5.2	实验一：24V 直流电磁铁	7
5.3	实验二：二维平行板电容器	8
5.4	实验三：二维变压器概念模型	8
5.5	实验四：二维电感器	9
5.6	实验结果汇总	9
6	如何复现实验	9
6.1	准备配置	9
6.2	检查环境	10
6.3	运行电磁铁实验	10
6.4	运行电容器实验	10
7	现阶段局限	11
8	结论	11

1 研究背景与问题

工业仿真软件的核心价值在于求解真实工程问题，但它的操作链条很长。以 Maxwell 为例，一个看似简单的电磁问题，也通常需要完成几何建模、材料设置、激励设置、边界条件、求解器配置、后处理和结果判断。对不熟悉软件的人来说，真正困难的不是“点哪个按钮”，而是如何把工程需求翻译成软件能够执行的仿真任务。

本项目要解决的问题可以概括为一句话：

能否让用户用自然语言描述工程目标，由智能体自动生成并执行 Maxwell 仿真，并告诉用户结果是否满足需求？

这里的“自动”不是让大模型凭空写出 Maxwell 工程文件。更可行的路线是：让大模型负责理解需求、形成仿真规格和修改策略；让本地程序负责校验、执行、调用 Maxwell 和保存工程。换句话说，大模型负责“想清楚要做什么”，Maxwell 负责“真实地算出来”，本地执行器负责“把二者可靠地连接起来”。

2 总体思路

2.1 为什么不直接让大模型生成 .aedt 文件

.aedt 是 Maxwell 的工程文件，不适合作为大模型直接生成的目标。原因有三点：

- 文件结构与软件版本、内部对象和求解器状态密切相关，直接生成字节流不稳定。
- 即使生成了文件，也很难保证 Maxwell 能正确打开、求解和后处理。
- 工程文件应该由 Maxwell 自己保存，这样版本兼容性和对象关系更可靠。

因此，本项目采用间接生成方式：

自然语言需求 → 仿真规格 → PyAEDT 脚本 → Maxwell 执行 → .aedt

这种路线的关键是：大模型不直接伪造工程文件，而是生成能够驱动 Maxwell 建模和求解的中间结果。

2.2 核心闭环

系统的主流程如下：

1. 用户输入中文工程需求。

2. 系统提取任务目标、几何参数、材料、激励、约束和需要输出的结果。
3. 大模型生成仿真规格和执行计划。
4. 系统生成或修复 PyAEDT 脚本。
5. 本地调用 Maxwell 完成建模、求解和保存工程。
6. 系统读取结果，判断每条数值约束是否满足。
7. 若不满足，将结果和失败原因反馈给大模型，生成下一轮修改方案。

用更形式化的方式表示，用户需求记为 R ，系统先得到仿真规格：

$$\Sigma = \{G, M, E, B, S, Q, C\}$$

其中 G 表示几何， M 表示材料， E 表示激励， B 表示边界， S 表示求解器， Q 表示待提取结果， C 表示约束。随后生成执行计划：

$$P = \{a_1, a_2, \dots, a_n\}$$

其中每个 a_i 是一项 Maxwell 操作，例如创建几何、赋材料、施加电流、建立求解设置或导出结果。执行后得到结果 Y 和评价 V 。如果 V 表示失败，则进入反馈更新：

$$\Sigma_{k+1}, P_{k+1} = \text{LLM}(R, \Sigma_k, P_k, Y_k, V_k)$$

当评价通过、达到最大迭代次数或无法修正时停止。

3 系统方法

3.1 自然语言到仿真规格

用户输入通常是非结构化的，例如：

做一个 24V 直流电磁铁，气隙 2mm，电流不超过 2A，尽量提高吸力，外形不要太大。

这句话包含了不同层次的信息：

- 数值条件：24V、2mm、不超过 2A。
- 设计目标：尽量提高吸力。

- 软约束：外形不要太大。
- 隐含信息：需要选择电磁铁结构、线圈参数、铁芯尺寸和求解方式。

系统会把这些内容整理为统一的仿真规格。仿真规格不是最终工程文件，而是一个机器可读的任务说明。它回答的问题是：需要做什么几何、用什么材料、加什么激励、跑什么求解器、提取什么结果、检查什么约束。

3.2 仿真规格到 PyAEDT 脚本

仿真规格形成后，系统需要把它变成 Maxwell 能执行的动作。本项目使用 PyAEDT 作为执行接口。PyAEDT 可以用 Python 控制 AEDT/Maxwell，例如创建二维模型、设置材料、施加激励、运行求解器和保存工程。

脚本生成过程包含两层保护：

- 静态检查：检查脚本是否有入口函数、是否包含 Maxwell 调用、是否存在危险操作。
- 运行隔离：生成脚本在独立 Python 进程中运行，避免脚本异常影响主程序。

这样做的目的不是追求形式复杂，而是保证“自动生成脚本”这件事可控。大模型可以参与生成和修复脚本，但本地系统必须负责安全边界和执行稳定性。

3.3 结果评价

工业仿真的重点不是“软件是否跑完”，而是“需求是否满足”。因此系统把结果分成两层判断：

- 执行状态：Maxwell 是否完成建模、求解和导出。
- 约束状态：用户提出的数值条件是否满足。

例如在电磁铁问题中，如果用户要求 24V 且电流不超过 2A，系统不会只看 Maxwell 是否运行成功，还会估算线圈电阻和 24V 下的工作电流。若电阻为 R_{coil} ，供电电压为 V ，则估算电流为：

$$I_{\text{est}} = \frac{V}{R_{\text{coil}}}$$

若要求 $I_{\text{est}} \leq I_{\text{max}}$ ，则等价于：

$$R_{\text{coil}} \geq \frac{V}{I_{\text{max}}}$$

这使得“24V”和“不超过 2A”不再只是文本条件，而是可计算、可判定的工程约束。

3.4 反馈修正

当结果不满足约束时，系统不会简单报错结束，而是把失败原因反馈给大模型。例如首轮设计可能得到：

- 供电电压：24V。
- 电流上限：2A。
- 估算电流：14.9A。
- 失败原因：24V 下电流远超上限。

大模型收到的不是一句“失败了”，而是结构化反馈，包括当前设计、仿真输出、约束判定和未满足项。随后它生成下一轮设计修改，例如增加匝数、调整线圈尺寸或改变电流设定。系统再用新设计重新生成脚本、重新运行 Maxwell、重新评价结果。

这就是当前系统区别于普通脚本自动化的核心：它不是只跑一次，而是具有“结果反馈到需求修正”的闭环。

4 已实现任务族

当前系统已经验证了四类任务族。这里的“验证”是指可以生成 .aedt 工程、执行 Maxwell、导出结果并完成评价。

任务族	当前能力	说明
二维电磁铁	建模、磁静态求解、最大磁密提取、电压电流约束评价、反馈修正	当前最完整的闭环任务
二维电容器	平行板几何、静电求解、电场和电容提取	用于验证系统跨物理场能力
二维变压器	概念几何、匝比、电压估算、磁密提取	属于概念验证模型，不等于完整暂态耦合模型
二维电感器	磁静态求解、磁密提取、等效电感估算	属于 Maxwell 结果与解析估算结合

这四类任务说明系统已经从单一电磁铁任务扩展到多任务族，但还不能宣称支持任意 Maxwell 问题。更准确的表述是：系统骨架具备通用性，稳定落地能力正在逐类扩展。

5 实验设计与结果

5.1 实验设置

实验环境如下：

- 操作系统：Windows。
- 仿真软件：Ansys Electronics Desktop Student 2025 R2。
- 执行接口：PyAEDT。
- 大模型接口：Codexa 兼容接口。
- 默认模型：gpt-5.4。
- 推理强度：high。
- 最大反馈迭代次数：2。

每次实验都会在 `workspace` 下生成一个独立目录。该目录保存用户需求、仿真规格、执行计划、生成脚本、脚本检查结果、Maxwell 工程文件、输出结果和评价结果。因此实验不是只依赖网页显示，而是可以从文件层面复核。

5.2 实验一：24V 直流电磁铁

输入需求为：

做一个 24V 直流电磁铁，气隙 2mm，电流不超过 2A，尽量提高吸力，外形不要太大。

该任务的难点在于 24V 和 2A 是联合约束。系统必须保证线圈在 24V 供电时估算电流不超过 2A。最新验证结果如下：

- 最大磁密： 9.866×10^{-5} T。
- 估算线圈电阻：13.41 Ω 。
- 24V 下估算电流：1.79 A。
- 总体评价：通过。

这说明系统不仅完成了 Maxwell 求解，还完成了电气约束验证。此前首轮设计曾出现电流过大的情况，经过反馈修正后，系统将设计调整到约束范围内。

5.3 实验二：二维平行板电容器

输入需求为：

做一个二维平行板电容器，板间距 1mm，板宽 20mm，施加 100V 电压，求电场强度和电容。

结果如下：

- 施加电压：100 V。
- 板间距：1 mm。
- 最大电场结果：约 3.70×10^5 V/m。
- 电容结果：198.18 pF。
- 总体评价：通过。

该实验的意义在于证明系统不是只围绕电磁铁场景工作。它可以进入 Maxwell 的静电求解链路，并提取电容矩阵结果。

5.4 实验三：二维变压器概念模型

输入需求为：

做一个 10000V 到 220V 市电的变压器。

结果如下：

- 原边电压：10000 V。
- 目标副边电压：220 V。
- 原边匝数：500。
- 副边匝数：11。
- 匝比：0.022。
- 估算副边电压：220 V。
- 最大磁密： 2.224×10^{-5} T。
- 总体评价：通过。

需要说明的是，该实验是二维概念模型，主要验证“自然语言到变压器类 Maxwell 任务”的可执行链路。它还不是完整的工频变压器高保真设计工具，暂未覆盖损耗、温升、绝缘、复杂绕组和暂态耦合等工程细节。

5.5 实验四：二维电感器

电感器任务用于验证系统对磁路参数和等效电感估算的处理能力。最新结果如下：

- 电流：2.0 A。
- 线圈匝数：600。
- 估算电感：0.326 H。
- 最大磁密： 9.029×10^{-5} T。
- 总体评价：通过。

该任务同样属于 Maxwell 计算与简化工程估算结合的验证。它说明系统可以围绕不同输出目标组织仿真过程，而不是固定只输出磁密。

5.6 实验结果汇总

实验任务	关键结果	评价
24V 直流电磁铁	$B_{\max} = 9.866 \times 10^{-5}$ T; $R = 13.41 \Omega$; 24V 下 $I = 1.79$ A	通过
二维电容器	电容 198.18 pF; 最大电场约 3.70×10^5 V/m	通过
二维变压器	匝比 0.022; 估算副边电压 220 V; $B_{\max} = 2.224 \times 10^{-5}$ T	通过
二维电感器	估算电感 0.326 H; $B_{\max} = 9.029 \times 10^{-5}$ T	通过

6 如何复现实验

为了让他人能够复现，本节给出最小操作流程。这里不展开源代码，只说明运行步骤和成功判据。

6.1 准备配置

进入项目根目录，复制示例配置：

```
cd F:\maxwell_agent_project
Copy-Item .env.example .env
```

然后在 `.env` 中填入自己的 API key，并确认模型设置为：

```
CODEXA_MODEL=gpt-5.4
CODEXA_REASONING_EFFORT=high
DESIGN_FEEDBACK_MAX_ITERS=2
```

6.2 检查环境

```
.\.venv\Scripts\python.exe -m maxwell_agent.cli probe-env
.\.venv\Scripts\python.exe -m maxwell_agent.cli smoke-llm
```

第一条命令用于确认 Maxwell 和 PyAEDT 是否可用，第二条命令用于确认云端模型是否可用。

6.3 运行电磁铁实验

```
.\.venv\Scripts\python.exe -m maxwell_agent.cli run "做一个24V直流电磁铁，气隙2mm，电流不超过2A，尽量提高吸力，外形不要太大。"
```

成功后，在新生成的 `workspace` 目录中应看到：

- `electromagnet_2d_attempt_*.aedt`
- `outputs.json`
- `evaluation.json`
- 如发生反馈修正，还会有 `intake_feedback_01.json` 和第二轮工程文件。

判断成功不是看命令有没有结束，而是看 `evaluation.json` 中 `overall_status` 是否为 `passed`。

6.4 运行电容器实验

```
.\.venv\Scripts\python.exe -m maxwell_agent.cli run "做一个二维平行板电容器，板间距1mm，板宽20mm，施加100V电压，求电场强度和电容。"
```

成功判据为：

- 生成 `capacitor_2d_attempt_01.aedt`。
- `outputs.json` 中包含电容和电场结果。
- `evaluation.json` 为通过。

7 现阶段局限

当前系统已经具备从自然语言到 Maxwell 工程文件的闭环，但仍处于原型阶段。主要局限包括：

- 稳定支持的任务族仍然有限，目前主要是四类二维或概念模型。
- 变压器和电感器当前偏概念验证，尚未覆盖完整工程级建模细节。
- 复杂三维结构、旋转电机、多物理场耦合和瞬态问题仍需继续扩展执行能力。
- 大模型反馈修正可以提升自动化程度，但不能保证任意约束组合都一定收敛。
- Student 版 Maxwell 对并发和进程释放较敏感，因此当前网页端采用单任务运行策略。

这些局限并不否定当前结果。更准确地说，当前成果证明了“自然语言需求到工业仿真执行结果”的技术路线可行，但还不能把它包装成任意 Maxwell 问题都能解决的通用产品。

8 结论

本项目完成了一个面向 Ansys Maxwell 的工业软件智能体原型。系统能够接收中文需求，将其转化为仿真规格和执行计划，生成并检查 PyAEDT 脚本，在本地调用 Maxwell 生成 .aedt 工程文件，并根据结果判断约束是否满足。对于电压和电流等数值约束冲突，系统已经验证了“先仿真、再反馈、再修正、再仿真”的闭环。

从实验结果看，系统已经覆盖二维电磁铁、二维电容器、二维变压器概念模型和二维电感器概念模型四类任务。当前最重要的价值不是某一个单独算例，而是建立了一条可扩展的技术路线：把大模型用于需求理解、规格生成和反馈修正，把 Maxwell 用于真实物理求解，把本地执行器用于安全执行和结果校核。

下一步工作应集中在三方面：第一，扩展更多 Maxwell 操作原语和后处理能力；第二，提高复杂任务的反馈修正稳定性；第三，将概念模型逐步升级为更接近工程实际的高保真模型。